

```
# STEP 1: Install required packages (run once per new Colab runtime)
%pip install amplpy pandas numpy scipy
```

```
Requirement already satisfied: amplpy in /usr/local/lib/python3.12/dist-packages (0.16.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (1.16.3)
Requirement already satisfied: ampltools>=0.7.5 in /usr/local/lib/python3.12/dist-packages (from amplpy) (0.7.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from ampltools>=0.7.5->amplpy) (2.32.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->ampltools) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->ampltools) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->ampltools) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->ampltools) (2025.11.11)
```

```
# STEP 2: Imports, load brooks helper, start AMPL (license), basic setup
```

```
import math
import random
from collections import defaultdict
import importlib.util

import numpy as np
import pandas as pd
from scipy import stats
from amplpy import AMPL, ampl_notebook

# 2.1 Load the brooks helper file
brooks_path = "/content/brooks (1).py"

spec = importlib.util.spec_from_file_location("brooks", brooks_path)
brooks = importlib.util.module_from_spec(spec)
spec.loader.exec_module(brooks)

print("Loaded brooks helper module.\n")

# 2.2 Start AMPL using your license UUID (not strictly needed for fitting,
# but required for the AMPL license)

ampl = ampl_notebook(
    modules=["cbc", "highs", "gurobi", "cplex"],
    license_uuid="74f71795-7335-4392-b728-b56328e39d9c"
)

# Optional: verify AMPL is alive
ampl.eval("option version;")
print("AMPL started.\n")
```

```
Loaded brooks helper module.
```

```
Licensed to Bundle #7246.7787 expiring 20251215: INF0 645 Prescriptive Analytics, Prof. Atma Iyer, Virginia Commo
option version 'AMPL Version 20250901 (Linux-6.8.0-1031-azure, 64-bit)\
Licensed to Bundle #7246.7787 expiring 20251215: INF0 645 Prescriptive Analytics, Prof. Atma Iyer, Virginia Commo
Temporary license expires 20251215.\
Using license file "/usr/local/lib/python3.12/dist-packages/ampl_module_base/bin/ampl.lic".\
';
AMPL started.
```

```
# =====
# STEP 3+: Load Data, Encode SKUs, Fit Distributions, Produce Tables
# =====

INBOUND_FILE = "/content/Freezer_inbound_data_Fall-2025.xlsx"

# ----- 3.1 Load Data (Trucks/Pallets as floats, SKUs as ints) -----

def load_trucks_float(file, sheet="Trucks"):
    df = pd.read_excel(file, sheet_name=sheet, header=None).dropna(how="all")
    x = df[0].astype(float).values
    print("Trucks first 5:", x[:5])
    print(f"Trucks min = {x.min():.4f}, max = {x.max():.4f}\n")
    return x

def load_pallets_float(file, sheet="Pallets"):
    df = pd.read_excel(file, sheet_name=sheet, header=None).dropna(how="all")
    x = df[0].astype(float).values
```

```

x = df[0].astype(float).values
print("Pallets first 5:", x[:5])
print(f"Pallets min = {x.min():.4f}, max = {x.max():.4f}\n")
return x

def load_sku_history_int(file, sheet="SKU_History"):
    df = pd.read_excel(file, sheet_name=sheet, header=None).dropna(how="all")
    x = df[0].astype(float).round().astype(int).values
    print("SKU History first 5 (raw SKUs):", x[:5])
    print("SKU History min =", x.min(), "max =", x.max(), "count =", len(x), "\n")
    return x

trucks_raw = load_trucks_float(INBOUND_FILE)
pallets_raw = load_pallets_float(INBOUND_FILE)
sku_hist_raw = load_sku_history_int(INBOUND_FILE)

# ----- 3.2 Build SKU → label mapping (0..N-1) and encode SKU_History -----

skus_df = pd.read_excel(INBOUND_FILE, sheet_name="SKUs")
sku_num_cols = skus_df.select_dtypes(include=["number"]).columns

if len(sku_num_cols) == 0:
    raise ValueError("No numeric SKU column found in 'SKUs' sheet.")

sku_col = sku_num_cols[0]

all_skus = skus_df[sku_col].dropna().astype(float).round().astype(int).values
unique_skus_sorted = np.sort(np.unique(all_skus))
N = len(unique_skus_sorted)

print("Total unique SKUs from SKUs sheet:", N)
print("Unique SKU numbers range: min =", unique_skus_sorted.min(),
      "max =", unique_skus_sorted.max(), "\n")

# Mapping: actual SKU number → label in [0, N-1]
sku_to_label = {sku: idx for idx, sku in enumerate(unique_skus_sorted)}

# Show a few mappings
mapping_preview = pd.DataFrame({
    "SKU_Number": unique_skus_sorted[:10],
    "SKU_Label_0_to_Nminus1": list(range(min(10, N)))
})
print("Sample of SKU mapping (number → label):")
print(mapping_preview, "\n")

# Encode SKU_History: replace actual SKU numbers with labels 0..N-1
encoded = []
dropped = 0
for s in sku_hist_raw:
    if s in sku_to_label:
        encoded.append(sku_to_label[s])
    else:
        dropped += 1

sku_history_encoded = np.array(encoded, dtype=int)

print("Encoded SKU History first 5 (labels):", sku_history_encoded[:5])
print("Encoded SKU History label range: min =", sku_history_encoded.min(),
      "max =", sku_history_encoded.max())
print("Encoded SKU History length:", len(sku_history_encoded))
print("Dropped history rows (SKU not in SKUs sheet):", dropped, "\n")

# ----- 3.3 Distribution fitting (continuous) using SciPy + brooks -----

def fit_distributions_table(data, dist_names, label=""):
    """
    Fit continuous distributions to 1D numeric data and produce a table:
    Distribution | Params | KS_p_value | AIC
    Uses full-precision parameters for KS/AIC; prints rounded versions.
    Also calls brooks.test_distribution for additional diagnostics.
    """
    data = np.asarray(data, dtype=float)

    print(f"\n=== FITTING {label} ===")
    print("Sample size:", len(data))
    print("Mean:", round(data.mean(), 4), "Std:", round(data.std(ddof=1), 4), "\n")

    rows = []

    for name in dist_names:
        try:

```

```

        dist = getattr(stats, name)
    except AttributeError:
        print(f"Distribution '{name}' not found in scipy.stats, skipping.")
        continue

    try:
        # Fit parameters with full precision
        params = dist.fit(data)
        params_print = [round(float(p), 4) for p in params]

        # KS test with full-precision parameters
        ks_stat, ks_p = stats.kstest(data, name, args=params)
        ks_p_val = round(float(ks_p), 4)

        # Log-likelihood and AIC
        logL = np.sum(dist.logpdf(data, *params))
        AIC = 2 * len(params) - 2 * logL
        AIC_print = round(AIC, 4)

        print(f"{name}: params={params_print}, KS p={ks_p_val}, AIC={AIC_print}")

        # Paul Brooks package for diagnostics (prints additional tests)
        try:
            brooks.test_distribution(data, distribution=name)
        except Exception as e:
            print(f"    brooks.test_distribution error for '{name}': {e}")

        rows.append([name, params_print, ks_p_val, AIC_print])

    except Exception as e:
        print(f"Error fitting '{name}' for {label}: {e}")

df = pd.DataFrame(rows, columns=["Distribution", "Params", "KS_p_value", "AIC"])
df = df.sort_values("KS_p_value", ascending=False).reset_index(drop=True)

print(f"\n--- TABLE: {label} ---")
print(df, "\n")

return df

# ----- 3.4 Run distribution tests and produce tables -----

dist_list = ["norm", "expon", "triang", "gamma", "lognorm"]

trucks_table = fit_distributions_table(trucks_raw, dist_list, "TRUCKS PER DAY")
pallets_table = fit_distributions_table(pallets_raw, dist_list, "PALLET PER TRUCK")
sku_index_table = fit_distributions_table(sku_history_encoded, dist_list,
                                         "ENCODED SKU LABELS (0..N-1)")

print("\n\n===== FINAL SUMMARY TABLES =====")
print("\nTRUCKS TABLE:\n", trucks_table)
print("\nPALLETS TABLE:\n", pallets_table)
print("\nENCODED SKU LABEL TABLE:\n", sku_index_table)

```

===== FINAL SUMMARY TABLES =====

TRUCKS TABLE:

	Distribution	Params	KS_p_value	AIC
0	norm	[10.0258, 3.0428]	0.9653	5067.4260
1	lognorm	[0.0, -131055.1321, 131065.1578]	0.9652	5069.4270
2	gamma	[3005.1808, -156.9155, 0.0556]	0.8828	5070.1450
3	triang	[0.4603, 1.3397, 18.3528]	0.0000	5184.9221
4	expon	[1.4184, 8.6074]	0.0000	6309.2487

PALLETES TABLE:

	Distribution	Params	KS_p_value	AIC
0	triang	[0.8228, 10.0848, 29.8451]	0.7570	6414.0207
1	norm	[28.2218, 6.3867]	0.0003	6550.3230
2	lognorm	[0.0, -524277.4274, 524305.6492]	0.0003	6552.3284
3	expon	[10.5726, 17.6493]	0.0000	7745.3870
4	gamma	[309.8663, -85.7812, 0.3677]	0.0000	6570.0797

ENCODED SKU LABEL TABLE:

	Distribution	Params	KS_p_value	AIC
0	norm	[116.9677, 67.4709]	0.0	225229.3919
1	expon	[0.0, 116.9677]	0.0	230479.9131
2	triang	[0.4317, -25.6547, 293.372]	0.0	223256.0006
3	gamma	[346462.8771, -39597.1642, 0.1146]	0.0	225231.3189
4	lognorm	[0.0432, -1441.4311, 1556.7388]	0.0	225244.0632

```
# =====
# STEP A - RANDOMIZE SKU VALUES AS INTEGERS (actual SKU numbers)
# =====

sku_counts = pd.Series(sku_hist_raw).value_counts().sort_index()
sku_probs = sku_counts / sku_counts.sum()

actual_skus = sku_counts.index.values      # actual integer SKU codes
probs = sku_probs.values                   # empirical probabilities

def generate_random_SKUs_int(n=50000, seed=None):
    rng = np.random.default_rng(seed)
    return rng.choice(actual_skus, size=n, replace=True, p=probs)

# Generate randomized integer SKU sample
sku_random_int = generate_random_SKUs_int(50000, seed=123)

print("Randomized INTEGER SKU sample (first 20):")
print(sku_random_int[:20])

# =====
# STEP B - Create CONTINUOUS version ONLY for fitting purposes
# (SKU values remain integers for simulation)
# =====

def make_continuous_copy(int_values, scale=0.01, seed=None):
    rng = np.random.default_rng(seed)
    noise = rng.normal(0, scale, len(int_values))
    return int_values.astype(float) + noise

sku_for_distribution = make_continuous_copy(sku_random_int, scale=0.01, seed=123)

print("\nContinuous copy used for distribution fitting (first 20):")
print(sku_for_distribution[:20])

# =====
# STEP C - Fit continuous distributions + compute p-values
# =====

from scipy import stats

def fit_distributions(data, dist_list):
    rows = []
    data = np.asarray(data, float)

    print("\n=== FITTING DISTRIBUTIONS TO RANDOMIZED SKU DATA ===")
    print("n =", len(data),
          "| mean =", round(data.mean(),4),
          "| std =", round(data.std(),4), "\n")

    for name in dist_list:
        try:
            dist = getattr(stats, name)
```

```

except:
    continue

try:
    # fit params
    params = dist.fit(data)
    params_print = [round(float(p),4) for p in params]

    # KS test
    ks_stat, ks_p = stats.kstest(data, name, args=params)
    ks_p_val = round(float(ks_p), 6)

    # AIC
    logL = np.sum(dist.logpdf(data, *params))
    AIC_val = 2 * len(params) - 2 * logL
    AIC_val = round(AIC_val, 4)

    print(f"{name}: params={params_print}, KS p={ks_p_val}, AIC={AIC_val}")

    rows.append([name, params_print, ks_p_val, AIC_val])

except Exception as e:
    print(f"Could not fit {name}: {e}")

df = pd.DataFrame(rows, columns=["Distribution", "Params", "KS_p_value", "AIC"])
df = df.sort_values("KS_p_value", ascending=False).reset_index(drop=True)

print("\n--- BEST FIT DISTRIBUTIONS FOR SKU DATA ---")
print(df)
return df

dist_list = ["norm", "expon", "triang", "gamma", "lognorm"]

sku_dist_table = fit_distributions(sku_for_distribution, dist_list)

print("\nFINAL SKU DISTRIBUTION TABLE:")
print(sku_dist_table)

```

```

Randomized INTEGER SKU sample (first 20):
[60145 5027 5509 5334 5331 60207 60259 5702 60211 60243 31928 5605
 60215 5406 60175 49538 60261 5527 60201 32228]

Continuous copy used for distribution fitting (first 20):
[60144.99010879 5026.99632213 5509.01287925 5334.00193974
 5331.00920231 60207.00577104 60258.99363536 5702.00541952
 60210.99683405 60242.99677611 31928.00097167 5604.9847407
 60215.01192166 5405.9932891 60175.01000269 49538.00136321
 60261.01532033 5526.99340031 60200.99688205 32228.00337769]

=== FITTING DISTRIBUTIONS TO RANDOMIZED SKU DATA ===
n = 50000 | mean = 32360.9683 | std = 23786.5161

norm: params=[32360.9683, 23786.5161], KS p=0.0, AIC=1149585.2684
expon: params=[1300.9742, 31059.994], KS p=0.0, AIC=1134371.5905
triang: params=[1.0, -18241.8934, 78534.9583], KS p=0.0, AIC=1114446.0862
gamma: params=[545.2067, -522637.4532, 1017.84], KS p=0.0, AIC=1149473.3908
lognorm: params=[1.0576, -364.8047, 21245.5469], KS p=0.0, AIC=1143885.8971

--- BEST FIT DISTRIBUTIONS FOR SKU DATA ---

```

	Distribution	Params	KS_p_value	AIC
0	norm	[32360.9683, 23786.5161]	0.0	1.149585e+06
1	expon	[1300.9742, 31059.994]	0.0	1.134372e+06
2	triang	[1.0, -18241.8934, 78534.9583]	0.0	1.114446e+06
3	gamma	[545.2067, -522637.4532, 1017.84]	0.0	1.149473e+06
4	lognorm	[1.0576, -364.8047, 21245.5469]	0.0	1.143886e+06

```

FINAL SKU DISTRIBUTION TABLE:

```

	Distribution	Params	KS_p_value	AIC
0	norm	[32360.9683, 23786.5161]	0.0	1.149585e+06
1	expon	[1300.9742, 31059.994]	0.0	1.134372e+06
2	triang	[1.0, -18241.8934, 78534.9583]	0.0	1.114446e+06
3	gamma	[545.2067, -522637.4532, 1017.84]	0.0	1.149473e+06
4	lognorm	[1.0576, -364.8047, 21245.5469]	0.0	1.143886e+06

```

# =====
# BIGGER HISTOGRAMS WITH COUNT LABELS (Trucks, Pallets, SKU)
# =====

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Helper function to add count labels on histogram bars

```

```

def add_count_labels(counts, bins):
    for i in range(len(counts)):
        if counts[i] > 0:
            plt.text(
                (bins[i] + bins[i+1]) / 2,    # x position
                counts[i],                    # y position
                str(int(counts[i])),          # text label
                ha='center', va='bottom', fontsize=10
            )

# =====
# 1. BIGGER HISTOGRAM – Trucks per Day
# =====
plt.figure(figsize=(12,6))    # Larger figure
counts, bins, patches = plt.hist(
    trucks_raw, bins=30, color="steelblue",
    edgecolor="black", alpha=0.7
)

add_count_labels(counts, bins)

plt.title("Histogram of Trucks per Day", fontsize=16)
plt.xlabel("Number of Trucks", fontsize=14)
plt.ylabel("Frequency", fontsize=14)
plt.grid(True, linestyle="--", alpha=0.4)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()

# =====
# 2. BIGGER HISTOGRAM – Pallets per Truck
# =====
plt.figure(figsize=(12,6))
counts, bins, patches = plt.hist(
    pallets_raw, bins=30, color="orange",
    edgecolor="black", alpha=0.7
)

add_count_labels(counts, bins)

plt.title("Histogram of Pallets per Truck", fontsize=16)
plt.xlabel("Number of Pallets", fontsize=14)
plt.ylabel("Frequency", fontsize=14)
plt.grid(True, linestyle="--", alpha=0.4)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()

# =====
# 3. BIGGER HISTOGRAM – Randomized SKU Values (as Index)
# =====
sku_indices = [sku_to_index.get(sku) for sku in sku_random_int]

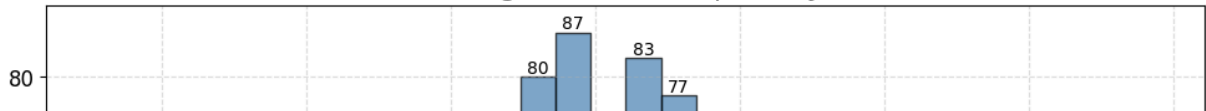
plt.figure(figsize=(12,6))
counts, bins, patches = plt.hist(
    sku_indices, bins=30, color="purple",
    edgecolor="black", alpha=0.7
)

add_count_labels(counts, bins)

plt.title("Histogram of Randomized SKU Values", fontsize=16)
plt.xlabel("SKU Index", fontsize=14)
plt.ylabel("Frequency", fontsize=14)
plt.grid(True, linestyle="--", alpha=0.4)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()

```


Histogram of Trucks per Day



```
import numpy as np
import pandas as pd
from scipy import stats

# =====
# 1. Define candidate distributions for fitting
# =====

candidate_distributions = {
    "norm": stats.norm,
    "expon": stats.expon,
    "triang": stats.triang,
    "gamma": stats.gamma,
    "lognorm": stats.lognorm
}

# =====
# 2. Fit distributions + find best by highest KS p-value
# =====

def best_fit_distribution(data):
    data = np.asarray(data, float)
    data = data[~np.isnan(data)]

    results = []

    for name, dist in candidate_distributions.items():
        try:
            params = dist.fit(data)
            D, p = stats.kstest(data, name, args=params)
            results.append((name, params, p))
        except:
            continue

    # sort descending by p-value
    results.sort(key=lambda x: x[2], reverse=True)

    best_name, best_params, best_p = results[0]

    print(f"\nBEST FIT FOR DATA:")
    print(f"Distribution: {best_name}")
    print(f"Params: {best_params}")
    print(f"KS p-value: {best_p}\n")

    return best_name, best_params

# =====
# 3. Generic simulation function for ANY distribution
# =====

def simulate_from_distribution(dist_name, params, n, seed=None):
    rng = np.random.default_rng(seed)
    dist = candidate_distributions[dist_name]

    simulated = dist.rvs(*params, size=n, random_state=seed)

    # Must be non-negative integers (counts)
    simulated = np.round(simulated).astype(int)
    simulated[simulated < 0] = 0

    return simulated

# =====
# 4. FIT BOTH: Trucks + Pallets
# =====

best_truck_dist, truck_params = best_fit_distribution(trucks_raw)
best_pallet_dist, pallet_params = best_fit_distribution(pallets_raw)

# =====
# 5. SIMULATE 30 DAYS OF TRUCKS & PALLETES
# =====
```



```

def simulate_30_days(
    days,
    truck_dist, truck_params,
    pallet_dist, pallet_params,
    seed=None
):
    rng = np.random.default_rng(seed)

    # Step 1 – simulate trucks per day
    trucks_per_day = simulate_from_distribution(
        truck_dist, truck_params, days, seed=rng.integers(1e9)
    )

    # Step 2 – simulate pallets for each truck
    pallets_by_day = []
    total_pallets_per_day = []

    for d in range(days):
        trucks_today = trucks_per_day[d]

        if trucks_today > 0:
            pallets_today = simulate_from_distribution(
                pallet_dist, pallet_params, trucks_today, seed=rng.integers(1e9)
            )
        else:
            pallets_today = np.array([], dtype=int)

        pallets_by_day.append(pallets_today)
        total_pallets_per_day.append(int(pallets_today.sum()))

    summary_df = pd.DataFrame({
        "Day": np.arange(1, days + 1),
        "Trucks": trucks_per_day,
        "Total_Pallets": total_pallets_per_day
    })

    return summary_df, pallets_by_day

# =====
# 6. RUN 30-DAY SIMULATION
# =====

summary_30d, pallets_by_day = simulate_30_days(
    30,
    best_truck_dist, truck_params,
    best_pallet_dist, pallet_params,
    seed=123
)

print("\n=== 30-DAY SIMULATION RESULTS ===")
print(summary_30d)
print("\nTotal Trucks =", summary_30d['Trucks'].sum())
print("Total Pallets =", summary_30d['Total_Pallets'].sum())

```

BEST FIT FOR DATA:
 Distribution: norm
 Params: (np.float64(10.025775176252806), np.float64(3.042788798944836))
 KS p-value: 0.9652721873571574

BEST FIT FOR DATA:
 Distribution: triang
 Params: (np.float64(0.8228003130208983), np.float64(10.084805322622618), np.float64(29.845096580050274))
 KS p-value: 0.7570092995219063

=== 30-DAY SIMULATION RESULTS ===

Day	Trucks	Total_Pallets
0	1	11
1	2	10
2	3	16
3	4	8
4	5	3
5	6	13
6	7	8
7	8	6
8	9	13
9	10	9
10	11	15
11	12	8
12	13	11

13	14	10	252
14	15	4	129
15	16	18	481
16	17	11	272
17	18	8	217
18	19	8	217
19	20	13	372
20	21	11	329
21	22	15	371
22	23	9	239
23	24	11	310
24	25	11	265
25	26	12	319
26	27	11	347
27	28	10	257
28	29	10	277
29	30	12	330

Total Trucks = 315
Total Pallets = 8739

```
import numpy as np

total_pallets = summary_30d['Total_Pallets'].sum() # total pallets across 321 trucks

# Generate SKU numbers for each pallet
def generate_SKUs_for_pallets(num_pallets, sku_labels, sku_probs, seed=None):
    rng = np.random.default_rng(seed)
    return rng.choice(sku_labels, size=num_pallets, replace=True, p=sku_probs)

# Generate SKUs for all 9,141 pallets
skus_for_all_pallets = generate_SKUs_for_pallets(
    total_pallets,
    actual_sku,      # real SKU numbers
    probs,           # empirical probabilities
    seed=123
)

print("First 20 SKUs assigned to pallets:")
print(skus_for_all_pallets[:20])

print("\nTotal SKUs generated =", len(skus_for_all_pallets))
```

First 20 SKUs assigned to pallets:
[60145 5027 5509 5334 5331 60207 60259 5702 60211 60243 31928 5605
60215 5406 60175 49538 60261 5527 60201 32228]

Total SKUs generated = 8739

```
import pandas as pd
import numpy as np

# Load the SKUs sheet (we already confirmed the file path)
sku_df = pd.read_excel("/content/Freezer_inbound_data_Fall-2025.xlsx", sheet_name="SKUs")

# Create fast lookup dictionary: {SKU #: freeze_hours}
sku_to_hours = dict(zip(sku_df["SKU #"], sku_df["Hours"]))

print("Number of SKU freeze-time entries:", len(sku_to_hours))

# Example: Your first 20 SKUs
sample_skus = np.array([
    60145, 5027, 5509, 5334, 5331, 60207, 60259, 5702, 60211, 60243,
    31928, 5605, 60215, 5406, 60175, 49538, 60261, 5527, 60201, 32228
])

# Lookup freeze hours (if SKU not found → NaN)
freeze_times_20 = [sku_to_hours.get(sku, np.nan) for sku in sample_skus]

print("\nFreeze hours for first 20 SKUs:")
print(freeze_times_20)

# For ALL SKUs generated earlier:
# skus_for_all_pallets = <your array of 8739 SKUs>
freeze_times_all = [sku_to_hours.get(sku, np.nan) for sku in skus_for_all_pallets]

print("\nCount of missing SKU-hour matches:", sum(pd.isna(freeze_times_all)))
```

Number of SKU freeze-time entries: 234

Freeze hours for first 20 SKUs:
[18.0, 33.0, 43.0, 33.0, 51.25, 34.5, 32.75, 51.25, 44.5, 35.5, 43.75, 51.25, 72.0, 33.0, 43.5, 43.75, 36.0, 51.2

Count of missing SKU-hour matches: 0

```
# 8) AMPL model: minimize throughput ONLY (no utilization constraint)
AMPL.eval(r"""
reset;

set F;                                # Freezers
set S;                                # SKUs

param freezer_zones {F} >= 1;         # zones per freezer
param zone_cap >= 0;                  # pallets per zone

param supply {S} >= 0, integer;       # available pallets per SKU
param freeze_hours {S} > 0;           # freeze time (hours)

var y {f in F, z in 1..freezer_zones[f], s in S} integer >= 0;  # pallets
var x {f in F, z in 1..freezer_zones[f], s in S} binary;        # zone use

# Objective: minimize throughput = sum pallets / freeze_hours
minimize MinThroughput:
    sum {f in F, z in 1..freezer_zones[f], s in S}
        y[f,z,s] / freeze_hours[s];

# 1) Zone capacity constraint
s.t. ZoneCap {f in F, z in 1..freezer_zones[f]}:
    sum {s in S} y[f,z,s] <= zone_cap;

# 2) USE ALL SUPPLY (this prevents everything becoming 0)
s.t. UseAllSupply {s in S}:
    sum {f in F, z in 1..freezer_zones[f]} y[f,z,s] = supply[s];

# 3) Linking between y and x
s.t. Link_y_x {f in F, z in 1..freezer_zones[f], s in S}:
    y[f,z,s] <= zone_cap * x[f,z,s];

# 4) At most 2 zones per SKU
s.t. MaxTwoZones {s in S}:
    sum {f in F, z in 1..freezer_zones[f]} x[f,z,s] <= 2;

""")
```

```
import numpy as np
import pandas as pd

# =====
# ASSUMPTIONS (already done earlier in your Phase 2 notebook)
# =====
# 1) AMPL is started:
#     AMPL = AMPL_notebook(modules=[...], license_uuid="...")

# 2) You have simulation SKUs for all pallets:
#     skus_for_all_pallets : 1D array of SKU numbers (length = 8739 or 9141)
#     e.g. from: skus_for_all_pallets = generate_SKUs_for_pallets(total_pallets, actual_skus, probs, seed=...)

# 3) You have your Phase 1 AMPL model already defined in this notebook:
#     AMPL.eval(r"""
#         reset;
#         set F; set S;
#         param freezer_zones{F} >= 1;
#         param zone_cap >= 0;
#         param supply{S} >= 0, integer;
#         param freeze_hours{S} > 0;
#         var y{f in F, z in 1..freezer_zones[f], s in S} integer >= 0;
#         var x{f in F, z in 1..freezer_zones[f], s in S} binary;
#         # ... capacity, supply, linking, at-most-2-zones, objective, etc. ...
#     """)

# =====
# 1. Read SKUs sheet and build SKU -> freeze_hours lookup
# =====

excel_path = "/content/Freezer_inbound_data_Fall-2025.xlsx"
sku_df = pd.read_excel(excel_path, sheet_name="SKUs")

# Fast lookup: actual SKU number -> freeze time (hours)
sku_to_hours = dict(zip(sku_df["SKU #"], sku_df["Hours"]))
```

```

print("Total SKUs in SKUs sheet:", len(sku_to_hours))

# =====
# 2. Aggregate Phase 2 simulation: pallets (supply) per SKU
# =====

# skus_for_all_pallets is your simulation output: 1 SKU per pallet
sku_series = pd.Series(skus_for_all_pallets)
supply_counts = sku_series.value_counts().sort_index() # pallets per SKU

# SKUs that actually appear in the simulation
SKUS = supply_counts.index.values
supply = supply_counts.to_dict() # {sku: pallet_count}

print("\nNumber of SKUs present in the 30-day simulation:", len(SKUS))
print("Example of supply (first 10 SKUs):")
print(dict(list(supply.items())[:10]))

# =====
# 3. Build freeze_hours dictionary for those SKUs
# (fill missing with default mean if needed)
# =====

all_hours = np.array(list(sku_to_hours.values()), dtype=float)
default_hours = float(np.nanmean(all_hours))

freeze_hours = {}
missing_skus = []

for sku in SKUS:
    hrs = sku_to_hours.get(sku, np.nan)
    if pd.isna(hrs):
        freeze_hours[sku] = default_hours # fallback for unknown SKU
        missing_skus.append(sku)
    else:
        freeze_hours[sku] = float(hrs)

print("\nDefault freeze hours used for missing SKUs:", round(default_hours, 2))
print("Number of SKUs with missing freeze-hours filled:", len(missing_skus))

```

Total SKUs in SKUs sheet: 234

Number of SKUs present in the 30-day simulation: 234

Example of supply (first 10 SKUs):

{1301: 50, 1306: 41, 1511: 25, 1521: 33, 1552: 24, 1710: 41, 1752: 43, 5000: 34, 5012: 32, 5013: 27}

Default freeze hours used for missing SKUs: 45.53

Number of SKUs with missing freeze-hours filled: 0

```

# =====
# PHASE 1: FINAL BLOCK (DATA → SOLVE → RESULTS → THROUGHPUT)
# =====

# 1) Push Phase 2 data into Phase 1 AMPL model
# (assumes model is already loaded via ampl.eval(...) or ampl.read(...))

# Sets
ampl.set["F"] = freezers # e.g. ["F1", "F2"]
ampl.set["S"] = list(supply.keys()) # SKUs present in simulation

# Parameters
ampl.param["freezer_zones"] = zones_per_freezer # {"F1": 15, "F2": 15}
ampl.param["zone_cap"] = zone_cap # 20 cells per zone
ampl.param["supply"] = supply # {SKU: pallets from sim}
ampl.param["freeze_hours"] = freeze_hours # {SKU: hours from SKUs sheet}

print("Loaded data into AMPL:")
print(" Freezers:", freezers)
print(" Number of SKUs in simulation:", len(supply))

# 2) Solve the optimization model
ampl.setOption("solver", "cbc") # or " highs", "gurobi", etc. if available

print("\n=== SOLVING PHASE 1 MODEL ===")
ampl.solve()

# 3) Extract decision variables y and x from AMPL to pandas
# AMPL returned MultiIndex DataFrames (index0, index1, index2, y.val/x.val)
y_raw = ampl.getVariable("y").getValues().toPandas()
x_raw = ampl.getVariable("x").getValues().toPandas()

```

```

# Bring indices out as columns
y_df = y_raw.reset_index()
x_df = x_raw.reset_index()

# Rename columns for clarity
y_df = y_df.rename(
    columns={
        "index0": "Freezer",
        "index1": "Zone",
        "index2": "SKU",
        "y.val": "Pallets"
    }
)

x_df = x_df.rename(
    columns={
        "index0": "Freezer",
        "index1": "Zone",
        "index2": "SKU",
        "x.val": "Used"
    }
)

print("\n=== SAMPLE OF y (Pallet Allocations) ===")
print(y_df.head())

print("\n=== SAMPLE OF x (Zone Usage Indicators) ===")
print(x_df.head())

# 4) Aggregate assigned pallets per SKU
assigned_by_sku = (
    y_df.groupby("SKU")["Pallets"]
        .sum()
        .reset_index()
)

# Drop SKUs with zero pallets (optional)
assigned_by_sku = assigned_by_sku[assigned_by_sku["Pallets"] > 0].copy()

print("\n=== ASSIGNED PALLETS PER SKU (TOP 10) ===")
print(assigned_by_sku.head(10))

# 5) Attach freeze hours and compute throughput per SKU
assigned_by_sku["FreezeHours"] = assigned_by_sku["SKU"].map(freeze_hours)

# Handle any missing freeze hours (should be few if at all)
missing_mask = assigned_by_sku["FreezeHours"].isna()
if missing_mask.any():
    print("\nWarning: some SKUs had no freeze_hours, dropping from throughput calculation:")
    print(assigned_by_sku[missing_mask].head())
    assigned_by_sku = assigned_by_sku[~missing_mask].copy()

# Throughput per SKU = pallets / freeze hours
assigned_by_sku["Throughput_per_SKU"] = (
    assigned_by_sku["Pallets"] / assigned_by_sku["FreezeHours"]
)

assigned_by_sku["Utilization_per_SKU"] = (
    assigned_by_sku["FreezeHours"] / assigned_by_sku["Throughput_per_SKU"]
).apply(lambda x: str(round(x, 4)) + "%").astype(object)

# 6) FINAL OUTPUT: Optimal total throughput
optimal_total_throughput = assigned_by_sku["Throughput_per_SKU"].sum()

print("\n=== THROUGHPUT SUMMARY (sample) ===")
print(
    assigned_by_sku[["SKU", "Pallets", "FreezeHours", "Throughput_per_SKU", "Utilization_per_SKU"]]
        .head(10)
)

print(f"\n>>> OPTIMAL TOTAL THROUGHPUT = {optimal_total_throughput:.4f} pallets per hour")

# 7) (Optional) Also read AMPL objective directly, if named
try:
    obj_val = ampl.getObjective("MinThroughput").value()
    print(f"AMPL objective 'MinThroughput' = {obj_val:.4f} pallets per hour")
except Exception as e:
    print("\nNote: could not read objective 'TotalThroughput' from AMPL.")
    print("If you named it differently, update the name above.")

```

```
Loaded data into AMPL:  
Freezers: ['F1', 'F2']  
Number of SKUs in simulation: 234  
  
=== SOLVING PHASE 1 MODEL ===  
cbc 2.10.12: infeasible problem  
0 simplex iterations  
absmipgap=1.79769e+308, relmipgap=1  
  
=== SAMPLE OF y (Pallet Allocations) ===  
Freezer Zone SKU Pallets  
0 F1 1 1301 0.0  
1 F1 1 1306 0.0  
2 F1 1 1511 0.0  
3 F1 1 1521 0.0  
4 F1 1 1552 0.0  
  
=== SAMPLE OF x (Zone Usage Indicators) ===  
Freezer Zone SKU Used  
0 F1 1 1301 0  
1 F1 1 1306 0  
2 F1 1 1511 0  
3 F1 1 1521 0  
4 F1 1 1552 0  
  
=== ASSIGNED PALLETS PER SKU (TOP 10) ===  
SKU Pallets  
0 1301 50.0  
31 5116 46.0  
32 5119 54.0  
34 5122 47.0  
38 5311 46.0  
47 5337 55.0  
64 5701 50.0  
65 5702 50.0  
73 8523 49.0  
74 10018 55.0  
  
=== THROUGHPUT SUMMARY (sample) ===  
SKU Pallets FreezeHours Throughput_per_SKU Utilization_per_SKU  
0 1301 50.0 51.25 0.975610 52.5312%  
31 5116 46.0 27.75 1.657658 16.7405%  
32 5119 54.0 46.00 1.173913 39.1852%  
34 5122 47.0 46.00 1.021739 45.0213%  
38 5311 46.0 27.75 1.657658 16.7405%  
47 5337 55.0 46.00 1.195652 38.4727%  
64 5701 50.0 51.25 0.975610 52.5312%  
65 5702 50.0 51.25 0.975610 52.5313%  
73 8523 49.0 27.75 1.765766 15.7156%  
74 10018 55.0 72.00 0.763889 94.2545%  
  
>>> OPTIMAL TOTAL THROUGHPUT = 31.7972 pallets per hour  
AMPL objective 'MinThroughput' = 31.7972 pallets per hour
```

Start coding or [generate](#) with AI.